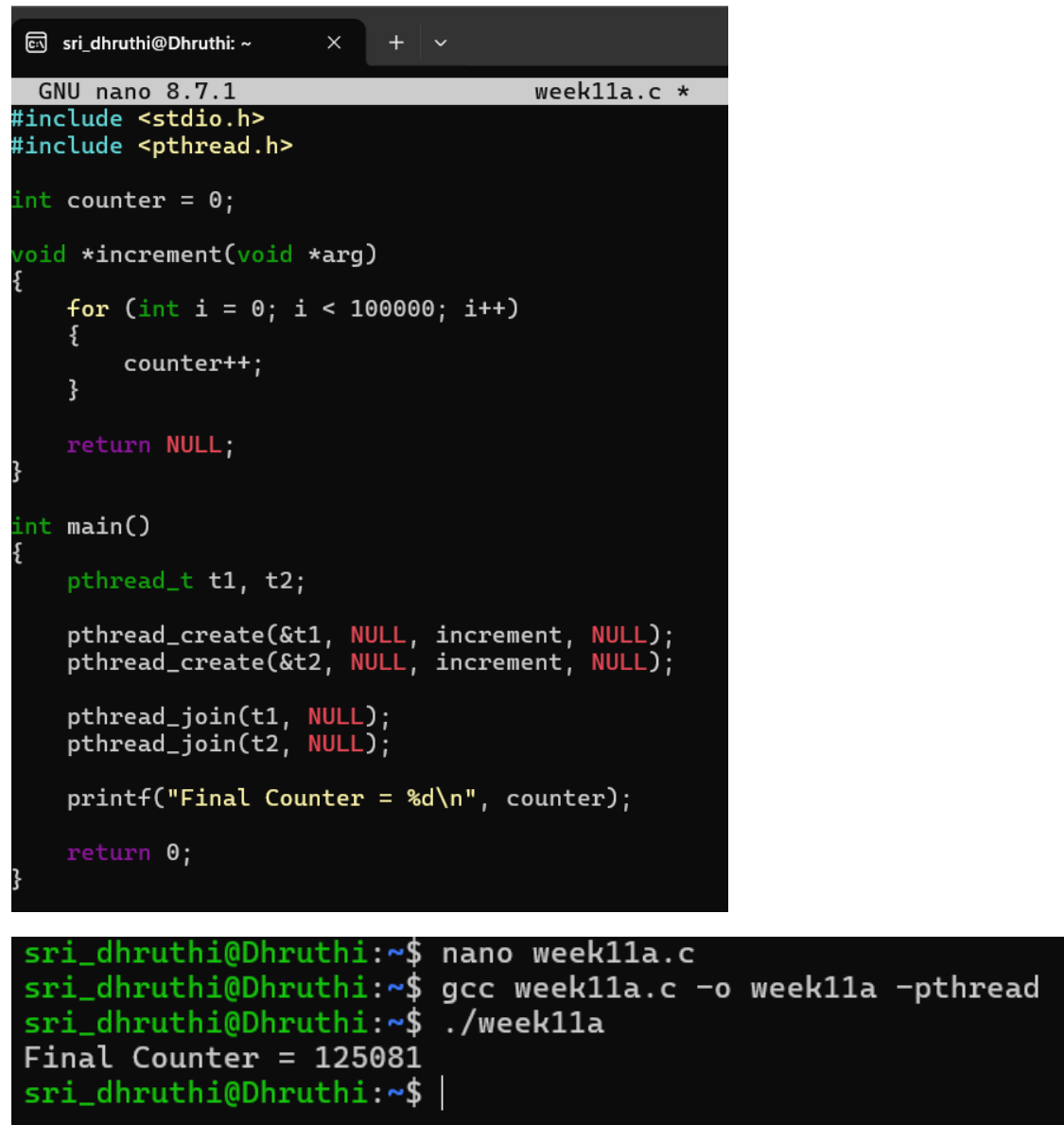


WEEK 11

Week 11A — Multithreaded Counter

This program creates multiple threads using `pthread_create()` and waits for them using `pthread_join()`.



```
sri_dhruthi@Dhruthi: ~
GNU nano 8.7.1 week11a.c *
#include <stdio.h>
#include <pthread.h>

int counter = 0;

void *increment(void *arg)
{
    for (int i = 0; i < 100000; i++)
    {
        counter++;
    }

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final Counter = %d\n", counter);

    return 0;
}

sri_dhruthi@Dhruthi:~$ nano week11a.c
sri_dhruthi@Dhruthi:~$ gcc week11a.c -o week11a -pthread
sri_dhruthi@Dhruthi:~$ ./week11a
Final Counter = 125081
sri_dhruthi@Dhruthi:~$ |
```

Week 11B — Race Condition with Mutex

This program protects the shared counter using a mutex to avoid simultaneous access by multiple threads.

```
sri_dhruthi@Dhruthi:~$ nano week11b.c
sri_dhruthi@Dhruthi:~$ gcc week11b.c -o week11b -pthread
sri_dhruthi@Dhruthi:~$ ./week11b
Final Counter = 200000
sri_dhruthi@Dhruthi:~$ |
```

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;

pthread_mutex_t lock;

void *increment(void *arg)
{
    for (int i = 0; i < 100000; i++)
    {
        pthread_mutex_lock(&lock);

        counter++;

        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&lock, NULL);

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final Counter = %d\n", counter);

    pthread_mutex_destroy(&lock);
}
```

```
sri_dhruthi@Dhruthi:~$ nano week11b.c
sri_dhruthi@Dhruthi:~$ gcc week11b.c -o week11b -pthread
sri_dhruthi@Dhruthi:~$ ./week11b
Final Counter = 200000
sri_dhruthi@Dhruthi:~$ |
```

